Lesson 5.6 **Duration: 40 minutes**

Iteration Algorithms

Learning Objectives

- Differentiate between counted loops (FOR) and conditional loops (WHILE)
- Construct iterative algorithms to solve repetitive computation problems
- Identify and prevent infinite loops through proper termination conditions

Engage (5-7 minutes)

Ask students to count from 1 to 100 aloud. Stop them at 50 and ask: how did you know when to stop? You had a counter and a termination condition. Introduce that computers need explicit loop constructs because they cannot infer "enough"—every iteration must be precisely controlled.

Explore (10-12 minutes)

Students simulate loops manually: they receive a stack of cards with numbers and must sum all even numbers. First approach: process each card individually (FOR loop). Second approach: keep drawing until a card exceeds 100 (WHILE loop). Record the number of steps and discuss when each loop type is appropriate.

Explain (10-12 minutes)

Iteration algorithms repeat a block of code until a condition is met. FOR loops are count-controlled—they iterate a predetermined number of times, ideal when the iteration count is known in advance. WHILE loops are condition-controlled—they continue as long as a Boolean expression remains true, suitable when the number of iterations is unknown. Both require: initialization, a loop body, and a termination condition that eventually becomes false. Failure to ensure termination produces infinite loops.

Elaborate (8-10 minutes)

Analyze loop patterns: accumulators (summing values), counters (counting occurrences), sentinel-controlled loops (reading until a special value), and flag-controlled loops (searching until found). Examine real algorithms that rely on iteration: multiplying large numbers, approximating square roots with Newton's method, and simulating physical systems. Discuss how break and continue statements alter loop behavior.

Evaluate (5-8 minutes)

Students trace through iterative algorithms step-by-step, tracking variable values in a loop table. Then they write a program that finds all prime numbers up to N using nested loops. Assessment: correct initialization, body logic, and guaranteed termination. Debugging exercise: identify the bug in an intentionally flawed loop.